

COMPILERS

Table of Contents

CHAPTER 4: SYNTAX ANALYSIS 1

THEORIES..... 1

EXERCISES 1

CHAPTER 5: SYNTAX-DIRECTED TRANSLATIONS.....	5
THEORIES.....	5
EXERCISES	6
CHAPTER 6: INTERMEDIATE-CODE GENERATION	8
THEORIES.....	8
THEORIES & EXERCISES (TYPE CHECKING)	9
EXERCISES (OTHERS)	10
CHAPTER 8: CODE GENERATION.....	15
THEORIES.....	15
EXERCISES	17
CHAPTER 9: MACHINE-INDEPENDENT OPTIMIZATION	20
THEORIES.....	20
EXERCISES	22

CHAPTER 4

SYNTAX ANALYSIS

Theories

4.1	What are the error recovery strategies generally used by parser? [2006. Marks: 2] 1. Panic-Mode Recovery 2. Phrase-Level Recovery 3. Error Productions 4. Global Correction
4.2	What is ambiguous grammar? [In-course 1, 2008-2009. Marks: 1] A grammar is ambiguous if it can have more than one parse tree generating a given string of terminals.
4.3	What is meant by left recursion in a grammar? [In-course 2005-2006. Marks: 1] A grammar is left recursive if it has a nonterminal A such that there is a derivation $A^+ \Rightarrow A\alpha$ for some string α. For example: $A \rightarrow A\alpha \mid \beta$
4.4	What is the problem with a production with an immediate left-recursion in a grammar? How can we eliminate left recursion? [2005. Marks: 3] A production with an immediate left-recursion can cause recursive-descent and top-down predictive parsers to loop forever. Removing immediate left recursion: If $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ Then we can rewrite the grammar as below: $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$ $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon$
4.5	Can we create LR parser table from an ambiguous grammar? When or why would one prefer to use ambiguous grammar? Justify your answer with example. [2008, In-course 1, 2008- 2009. Marks: 1 + 3]
4.6	Give an unambiguous grammar that is not LR(1). [2006. Marks: 3]
4.7	Define LL(k) and LR(k) grammars. Write down the basic properties of LL(1) and LR(1) grammars. [2007. Marks: 2 + 4]

4.8	What is a synchronizing set? Write down the procedure for panic mode error recovery in LL parsing. [2005. Marks: 4]
4.9	What is the difference between LR(0) grammar and SLR grammar? [2008. Marks: 2]
4.10	What is $\in$ -production? How $\in$ -productions are handled in bottom-up parsing? Explain it from the grammar below: [2007. Marks: 4] $S \rightarrow aB$ $B \rightarrow \in x$
4.11	What are inadequate states? How do we try to resolve the problem associated with such states in SLR and LR(1) parsing? [2005. Marks: 4]
4.12	What do you understand by shift-reduce and reduce-reduce conflicts? Why these conflicts arise? Explain rules for resolving these conflicts in parsing table. [2007. Marks: 6]
4.13	Show that the construction of an LALR parser by merging LR(1) item sets with common core of a LR(1) grammar will not introduce any shift/reduce conflict. [2004. Marks: 4]

1

4.14	Write down the general structure of LR parsers. [2003. Marks: 4]
4.15	What does the characters L, A, L, R and 1 in the name LALR(1) parser stand for? [In course. Marks: 1]
4.16	Which of the LL(1), SLR(1) and LR(1) can parse the following grammar? Why? [In-course. Marks: 2.5] $S \rightarrow A \mid B$ $A \rightarrow b \mid c$ $B \rightarrow c \mid a$
4.17	What does a lexical analyzer do when prefixes of the input string matches more than one patterns? [2007, 2004. Marks: 2]

Exercises

4.1	Consider the grammar: $S \rightarrow (L) \mid a$ $L \rightarrow L, S \mid S$ i. Find parse trees for the sentences (a, (a,a)) and (a,((a,a),(a,a))). ii. Construct a leftmost derivation for each of the sentences in part (i). iii. Construct a rightmost derivation for each of the sentences in part (i). [2003. Marks: 6]
4.2	Eliminate ambiguities for the grammar: [In-course 1, 2008-2009. Marks: 5] $E \rightarrow E + E \mid E * E \mid (E) \mid id$
4.3	Eliminate left recursion from the following grammar: $S \rightarrow SX \mid SSb \mid XS \mid a$ $X \rightarrow Xb \mid Sa \mid b$ When does the above algorithm of eliminating left recursion fail? [2008. Marks: 3 + 2]

4.4	Consider the grammar with the set of terminals: $S \rightarrow (L) \mid a \mid b$ $L \rightarrow L, S \mid S$ (i) Remove left-recursion from the grammar and find the First and Follow sets for each non-terminal of the modified grammar. (ii) Write down a recursive descent parser (i.e. parsing algorithm) for the modified grammar. [2007. Marks: 5 + 5]
4.5	Consider the following grammar for arithmetic expressions. $E \rightarrow E + T \mid E - T \mid T$ $T \rightarrow T * F \mid T / F \mid F$ $F \rightarrow (E) \mid id$ i. Write the above grammar eliminating immediate left recursion. ii. Draw the simplified transition diagram for the grammar. [2005. Marks: 4]
4.6	Consider the grammar with the set of terminals { (,), , , a, b }: $S \rightarrow (L) \mid a \mid b$ $L \rightarrow L, S \mid S$ a. Remove left-recursion from the grammar and find the First(1) and Follow(1) sets for each non-terminal of the modified grammar. b. Construct the operator precedence relation table for the grammar. c. Find the right-most derivation in reverse for the string (a, (b, a)) and indicate the handles at each step. d. Draw the transition diagram of the grammar and simplify it (if possible). [2004. Marks: 3]

	+ 3 + 4 + 3 + 3] e. Write down a recursive descent parser for the modified grammar. [In-course 2005-2006. Marks: 5]
4.7	Left factor the following grammar: [2003. Marks: 3] $A \rightarrow AabcA \mid Aad \mid AabA \mid Ad$
4.8	Build an LL parsing table for the following grammar: $S \rightarrow (L) \mid a$ $L \rightarrow L, S \mid S$ Is this an LL(1) grammar? Justify your answer. [2008. Marks: 5 + 2]
4.9	Show that the following grammar is SLR but not LR(0). [2008. Marks: 6] $S \rightarrow A$ $A \rightarrow a A$ $A \rightarrow a$
4.10	With LALR (lookahead LR) parsing, we can reduce the number of states in an LR(1) parser. Justify the above statement using the following grammar: [2008. Marks: 8] $S \rightarrow XX$ $X \rightarrow aX$ $X \rightarrow b$

4.11	Consider the grammar: $S \rightarrow aSbS \mid bSaS \mid \epsilon$. Show that the grammar is ambiguous by constructing two different left most derivations for a string. [2006. Marks: 3]
4.12	Find the LL(1) parsing table for the following grammar. Show the first and follow sets for the non-terminals. [In-course. Marks: 4 + 4] $S \rightarrow aS \mid Ab$ $A \rightarrow XYZ \mid \epsilon$ $X \rightarrow cS \mid \epsilon$ $Y \rightarrow dS \mid \epsilon$ $Z \rightarrow eS$
4.13	Give a grammar with the following first and follow sets. Your grammar should have no epsilon productions. The non-terminals are X, Y, Z and the terminals are a, b, c, d, e. [In-course. Marks: 6] $X Y Z a b c d e F$ $\text{First } b, d, f b, d c, e a b c d e f$ $\text{Follow } \$ c, e a \$ b, d c, e c, e a \$$
4.14	Consider the following grammar: $X \rightarrow YaYb \mid ZbZa$ $Y \rightarrow b$ $Z \rightarrow a \mid \epsilon$ i. Give the parse tree for the string “babb”. ii. Show the sentential forms of each stage in a top-down leftmost derivation of “baa”. iii. Show the sentential forms of each stage in a bottom-up rightmost derivation of “babb”. iv. Work out the following sets: FIRST(X), FOLLOW(X), FOLLOW(Y). [2006. Marks: 2 + 3 + 3 + 3]

4.15	$X \rightarrow X + X \mid Y++$ $Y \rightarrow a$ i. For the above grammar, construct the LR(1) parse table. ii. Point out where shift-reduce conflict(s) occurs. Give example(s) of a string(s) for which the parser will face the conflict(s)? iii. If the shift-reduce conflict(s) is resolved in favor of shift, what is the associativity of the ‘+’ operator that corresponds to this choice? [2006. Marks: 6 + 3 + 1]
4.16	For the following grammar, construct the DFA, recognizing viable prefixes, that includes just those states of an LR(1) parser that are pushed on the stack during a parse of the input: (id id&. The set of terminals is { , &, (,), id } [2005. Marks: 8] $Bexp \rightarrow Bexp \mid \mid Btrm \mid Bexp \mid Btrm \mid$ $Bexp \rightarrow Bexp \mid Btrm$ $Bexp \rightarrow Btrm$ $Btrm \rightarrow Btrm \&\& Bfct \mid Btrm \& Bfct \mid Bfct$ $Bfct \rightarrow (Bepr) \mid id$

4.17	Consider the following grammar: $S \rightarrow aBX \mid Ay$ $A \rightarrow ab$ $B \rightarrow a \mid b$ Prove that this grammar is not LR(0), but is SLR(1).
4.18	Construct the LALR parse table for the following grammar that generates a subset of all possible regular expressions. Resolve shift/reduce or reduce/reduce conflicts (if occurs) using the usual precedence rules of the operators. The set of input symbols is $\{[, ., *, id\}$ [2004. Marks: 12] $R \rightarrow R \mid R \mid R.R \mid R^* \mid id$
4.19	Construct the LALR parser table for the following grammar. Show all the necessary steps. [2003. Marks: 12] $P \rightarrow PaQ \mid Q$ $Q \rightarrow QR \mid R$ $R \rightarrow Rb \mid c \mid d$
4.20	Construct the predictive parsing table for the following grammar: [In-course. Marks: 6] $E \rightarrow TA$ $A \rightarrow +TA \mid -TA \mid \in$ $T \rightarrow FB$ $B \rightarrow *FB \mid /FB \mid \in$ $F \rightarrow -S \mid S$ $S \rightarrow v \mid (E)$ Show that the following grammar is not SLR(1): [In-course. Marks: 4] $S \rightarrow Aa \mid bAc \mid dc \mid bda$ $A \rightarrow d$
4.21	Construct SLR parsing table for the following grammar. [In-course 1, 2008-2009. Marks: 10] $S \rightarrow AS \mid b$ $A \rightarrow SA \mid a$
4.22	The following grammar for if-then-else statements is proposed to remedy the dangling-else ambiguity: $\text{stmt} \rightarrow \text{if expr then stmt} \mid \text{matched_stmt}$ $\text{matched_stmt} \rightarrow \text{if expr then matched_stmt else stmt} \mid \text{other}$ Justify whether this grammar is ambiguous or not. If the grammar is still ambiguous,

	rewrite the grammar by removing dangling-else ambiguity. [2003. Marks: 6]
4.23	Construct the DFA, recognizing viable prefixes, that includes just those states of an LR(1) parser for the following grammar, that are pushed on the stack during a parse of the input $y+++y$. [In-course. Marks: 6] $S \rightarrow A$ $A \rightarrow A + A \mid B++$ $B \rightarrow y$

4.24	$E \rightarrow E \mid T \mid T$ $T \rightarrow TF \mid F$ $F \rightarrow F^* \mid F^+ \mid (E) \mid a \mid b$ i. Modify the grammar for LL(1) parser. ii. Find First and Follow for each non-terminal of the modified grammar. iii. What are the items of the state associated with the viable prefix E T (E in SLR parsing for the grammar in (i). Show why the items are valid for the state. [In-course. Marks: 4 + 6 + 6 + 4]
------	---

CHAPTER 5

SYNTAX-DIRECTED TRANSLATIONS

Theories

5.1	Define synthesized and inherited attributes. [2003. Marks: 2] A <i>synthesized attribute</i> for a nonterminal A at a parse-tree node N is defined by a semantic rule associated with the production at N. Note that the production must have A as its head. A synthesized attribute at node N is defined only in terms of attribute values at the children of N and at N itself. An <i>inherited attribute</i> for a nonterminal B at a parse-tree node N is defined by a semantic rule associated with the production at the parent of N. Note that the production must have B as a symbol in its body. An inherited attribute at node N is defined only in terms of attribute values at N's parent, N itself, and N's siblings.
5.2	What is L-attributed definition? “Every s-attributed definition is L-attributed” – Justify your answer. [In-course 2, 2008-2009. Marks: 2 + 2] An SDD is called L-attributed definition if each attribute associated with the production bodies is either: 1. Synthesized, or 2. Inherited, but with the rules limited as follows. Suppose that there is a production $A \rightarrow X_1X_2 \cdots X_n$, and that there is an inherited attribute $X_i.a$ computed by a rule associated with this production. Then the rule may use only: (a) Inherited attributes associated with the head A. (b) Either inherited or synthesized attributes associated with the occurrences of symbols $X_1, X_2, \dots, X_{i-1}$ located to the left of X_i. (c) Inherited or synthesized attributes associated with this occurrence of X_i itself, but only in such a way that there are no cycles in a dependency graph formed by the attributes of this X_i. An SDD is called s-attributed definition if every attribute is synthesized. On the other hand, L attributed definition can have inherited attributes besides synthesized attributes. Hence, every s attributed definition is also L-attributed.

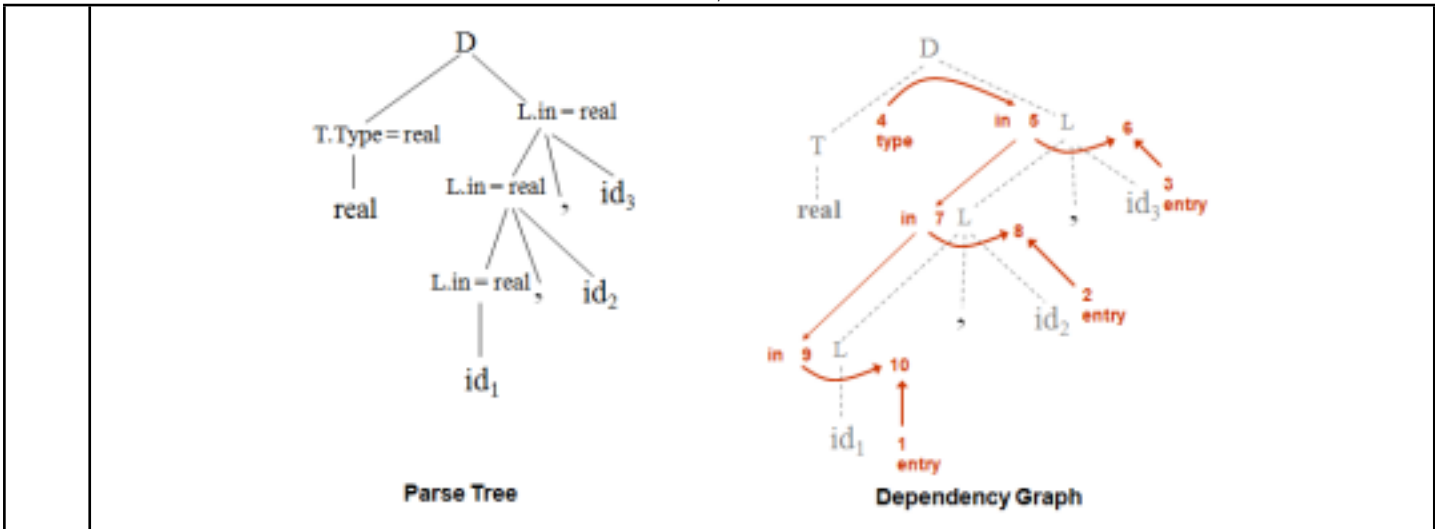
5.3	Explain how a translator for an s-attributed definition can be implemented as part of bottom-up parser. [In-course 2, 2008-2009. Marks: 3.5] When an SDD is s-attributed, its attributes can be evaluated in any bottom-up order of the nodes of the parse tree. It is often simple to evaluate the attributes by performing a post-order traversal of the parse tree and evaluating the attributes at a node N when the traversal leaves N for the last time. That is, the function postorder() defined below is applied to the root of the parse tree. <pre> postorder(N) { for (each child C of N, from the left) postorder(C); evaluate the attributes associated with node N; }</pre> S-attributed definition can be implemented during bottom-up parsing, since a bottom-up parser corresponds to a post-order traversal. Post-order corresponds exactly to the order in which an LR parser reduces a production body to its head.
-------------------	---

5.4	Write an algorithm for constructing a dependency graph from a given parse tree. [In-course 2, 2008-2009. Marks: 3] Put each semantic rule in to the form $b := f(c_1, c_2, \dots, c_k)$. <pre> for each node n in the parse tree do for each attribute a of the grammar symbol at n do construct a node in the dependency graph for a; for each node n in the parse tree do for each semantic rule $b := f(c_1, c_2, \dots, c_k)$ associated with the production used at n do for $i := 1$ to k do construct an edge from the node for c_i to the node for b;</pre>
-------------------	---

Exercises

5.1	Translate the arithmetic expression $a^*(b+c)$ into A syntax tree Postfix notation Three-address code [2003. Marks: 4] $*$ a minus $+$ $b \ c$ $t_1 = b + c$ $t_2 = \text{minus } t_1$ $t_3 = a * t_2$ $abc+-*$ Syntax Tree Postfix Notation 3-Address Code
-------------------	--

5.2	Consider the following grammar for declaration of identifiers. [2003. Marks: 4] $D \rightarrow TL$ $T \rightarrow \text{integer} \mid \text{real}$ $L \rightarrow L, \text{id} \mid \text{id}$ i. Write down a syntax directed definition to propagate the type information using inherited and synthesized attributes. ii. Rewrite the grammar so that the type information can be propagated using synthesized attributes only. <table border="1"> <thead> <tr> <th>PRODUCTION</th><th>SEMANTIC RULES</th></tr> </thead> <tbody> <tr> <td>1) $D \rightarrow TL$</td><td>$L.inh = T.type$</td></tr> <tr> <td>2) $T \rightarrow \text{int}$</td><td>$T.type = \text{integer}$</td></tr> <tr> <td>3) $T \rightarrow \text{float}$</td><td>$T.type = \text{float}$</td></tr> <tr> <td>4) $L \rightarrow L_1, \text{id}$</td><td>$L_1.inh = L.inh$ $addType(id.entry, L.inh)$</td></tr> <tr> <td>5) $L \rightarrow \text{id}$</td><td>$addType(id.entry, L.inh)$</td></tr> </tbody> </table> i.	PRODUCTION	SEMANTIC RULES	1) $D \rightarrow TL$	$L.inh = T.type$	2) $T \rightarrow \text{int}$	$T.type = \text{integer}$	3) $T \rightarrow \text{float}$	$T.type = \text{float}$	4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addType(id.entry, L.inh)$	5) $L \rightarrow \text{id}$	$addType(id.entry, L.inh)$
PRODUCTION	SEMANTIC RULES												
1) $D \rightarrow TL$	$L.inh = T.type$												
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$												
3) $T \rightarrow \text{float}$	$T.type = \text{float}$												
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addType(id.entry, L.inh)$												
5) $L \rightarrow \text{id}$	$addType(id.entry, L.inh)$												
5.3	A syntax directed definition for declaring identifiers is <table border="0"> <tr> <td><u>Productions</u></td><td><u>Semantic Actions</u></td></tr> <tr> <td>$D \rightarrow TL$</td><td>$L.in := T.type$</td></tr> <tr> <td>$T \rightarrow \text{int}$</td><td>$T.type := \text{integer}$</td></tr> <tr> <td>$T \rightarrow \text{real}$</td><td>$T.type := \text{real}$</td></tr> <tr> <td>$L \rightarrow L1, \text{id}$</td><td>$L1.in := L.in,$ $addtype(id.entry, L.in)$</td></tr> <tr> <td>$L \rightarrow \text{id}$</td><td>$addtype(id.entry, L.in)$</td></tr> </table> i. Draw an annotated parse tree for the sentence real id₁, id₂, id₃ ii. Draw the dependency graph for the parse tree in (i). [In-course 2, 2008-2009. Marks: 3 + 3]	<u>Productions</u>	<u>Semantic Actions</u>	$D \rightarrow TL$	$L.in := T.type$	$T \rightarrow \text{int}$	$T.type := \text{integer}$	$T \rightarrow \text{real}$	$T.type := \text{real}$	$L \rightarrow L1, \text{id}$	$L1.in := L.in,$ $addtype(id.entry, L.in)$	$L \rightarrow \text{id}$	$addtype(id.entry, L.in)$
<u>Productions</u>	<u>Semantic Actions</u>												
$D \rightarrow TL$	$L.in := T.type$												
$T \rightarrow \text{int}$	$T.type := \text{integer}$												
$T \rightarrow \text{real}$	$T.type := \text{real}$												
$L \rightarrow L1, \text{id}$	$L1.in := L.in,$ $addtype(id.entry, L.in)$												
$L \rightarrow \text{id}$	$addtype(id.entry, L.in)$												



5.4 Consider the following SDD:

Productions Rules

$$T \rightarrow FT' \quad T'.inh = F.val$$

$$T.val = T'.syn$$

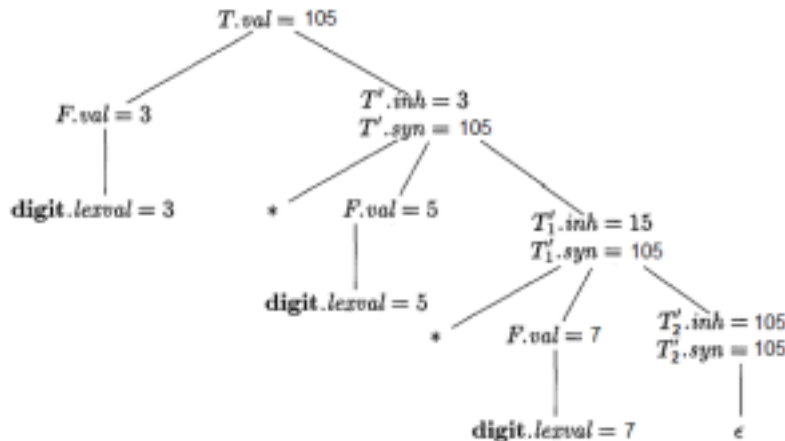
$$T' \rightarrow *FT_1' \quad T_1'.inh = T'.inh \times F.val$$

$$T'.syn = T_1'.syn$$

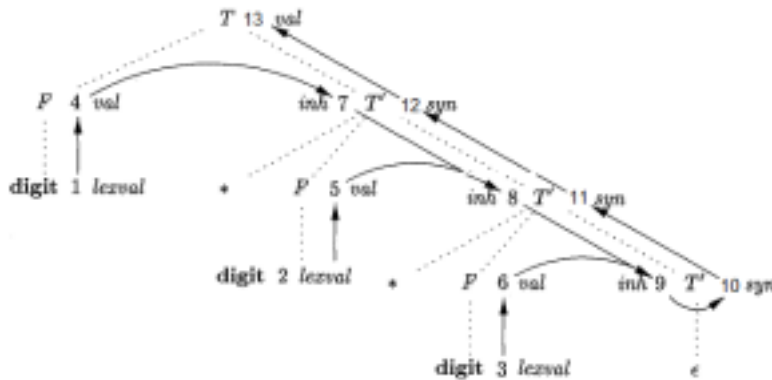
$$T' \rightarrow \epsilon \quad T'.syn = T'.inh$$

$$F \rightarrow \text{digit} \quad F.val = \text{digit.lexval}$$

- Draw the annotated parse tree for the expression $3*5*7$ using the semantic rules given above.
- Draw the dependency graph. [2008. Marks: 2 + 1]



i.



ii.

CHAPTER 6

INTERMEDIATE-CODE GENERATION

Theories

6.1 What are the advantages of using intermediate code generation? [In-course 2, 2008-2009. Marks: 2.5]

OR, Why would we be interested to generate intermediate code as the final product of the front end of a compiler? [2005. Marks: 2]

With a suitably defined intermediate representation, a compiler for language i and machine j can then be built by combining the front end for language i with the back end for machine j . This approach to creating suite of compilers can save a considerable amount of effort: $m \times n$ compilers can be built by writing just m front ends and n back ends.

6.2

Translate the expression $(a+b) * (c+d) + (b+c)$ into

i. Quadruples

ii. Triples

iii. Indirect triples [In-course 2, 2008-2009. Marks: 3]

Discuss the comparative advantages and disadvantages of the above representations. [2007. Marks: 2]

ALSO, What are the advantages and disadvantages of using indirect triple representation of 3-address codes? [2004. Marks: 2]

<u>op</u> <u>arg1</u> <u>arg2</u> <u>result</u>	<u>op</u> <u>arg1</u> <u>arg2</u>	<u>instruction</u>	<u>op</u> <u>arg1</u> <u>arg2</u>
$0 + a \ b \ t_1$	$0 + a \ b$	35 (0)	$0 + a \ b$
$t_1 = a + b$			
$1 + c \ d \ t_2$	$1 + c \ d$	36 (1)	$1 + c \ d$
$t_2 = c + d$			
$t_3 = t_1 * t_2$	$2 * t_1 \ t_2 \ t_3$	37 (2)	$2 * (0) \ (1)$
$t_4 = b + c$	$3 + b \ c \ t_4$	38 (3)	$3 + b \ c$
$t_5 = t_3 + t_4$	$4 + t_3 \ t_4 \ t_5$	39 (4)	$4 + (2) \ (3)$
		...	

3-Address Code Quadruples Triples Indirect Triples Comparative Advantages and

Disadvantages:

Advantages Disadvantages

Quadruples Instructions can be moved around without

requiring changing all references to it.

Takes more memory space than

Triples.

Triples Takes less memory space. Moving an instruction may require changing all references to it.

Instructions can be moved by reordering the

Takes more memory space than

Indirect

instruction list, without affecting the triples

Triples.

Triples

themselves. Thus, changes to all references to

	the moved instruction are not needed.
6.3	Find the expression that computes the relative address of the array element $A[i_1, i_2, \dots, i_n]$, where A is a $10 \times 10 \times \dots \times 10$ array stored in row-major form. [2006. Marks: 2]
6.4	What is backpatching? What is the advantage of backpatching? Explain with an example. [In-course 2, 2008-2009. Marks: 2 + 4] OR, Write short notes on backpatching. [2007. Marks: 3] Backpatching is the activity of filling up unspecified information of labels using appropriate semantic actions during the code generation process. In the code generation process, a separate pass is needed to bind jump labels to addresses.

	Backpatching merges the intermediate and target code generation into one pass.
--	--

Theories & Exercises (Type Checking)

6.1	Define static and dynamic type checking. [2006. Marks: 3]
6.2	What are type constructors? Name and explain the basic type constructors. [2003. Marks: 2 + 3]
6.3	What is name and structural equivalence of type expressions? [In-course 2, 2007. Marks: 1]
6.4	Write type expression for the following type: An array of pointers to reals, where array index ranges from 1 to 100. [2005. Marks: 2]
6.5	Show whether the following recursive type expressions are equivalent or not: [2005. Marks: 2]
6.6	How do we determine the structural equivalence to types? [In-course 2, 2007. Marks: 2] ALSO, Give an algorithm for testing structural equivalence of type expressions and explain how it works? [2003. Marks: 3]
6.7	How is encoding of type expressions used for checking structural equivalence of type expressions and what are its advantages (if any)? [2005, 2003; In-course 2, 2007. Marks: 3]
6.8	Consider the following grammar in a programming language. Here, P, D, T, S and E represent the program, the declarations, types, statements and expressions respectively. <pre> P → D; S D → D; D id : T T → char int array[num] of T S → id = E if E then S S; S E → literal num id E[E] </pre> Write down the syntax-directed translation for type checking and determine each declaration's offset in the activation record. Define the basic types, type constructs and functions you use. [2004. Marks: 7]

6.9

Let the following attribute grammar is used for type checking. [2008. Marks: 5]

```

E → num      { E.type := integer; }

E → id       { E.type := lookup(id.name); }

E → E1 + E2 { if (E1.type = integer & E2.type = integer)
                then E.type := integer;
                else type_error(); }

E → E1 [E2] { if (E2.type = integer & E1.type = array of T)
                then E.type := T;
                else type_error(); }

E → E1 (E2) { if (E1.type = T1 → T2 & E2.type = T1)
                then E.type := T2;
                else type_error(); }

E → E1 ^     { if (E1.type = ^T) then E.type := T
                else type_error(); }

```

Now

consider the following declarations:

```

I: integer;
A: array[20] of integer;
B: array[20] of ^integer;

```

10

F: ^integer → integer;

Show how type checking is performed for the following expressions using the grammar

- (i) **I := B[F(A[3])]**
- (ii) **I := A[F(B[3])]**

Exercises (Others)

6.1

Write an SDD to generate 3-address code for the following grammar:

 $P \rightarrow S$ $S \rightarrow \text{assign} \mid \text{if } (B) \ S_1 \mid S_1 \ S_2$ $B \rightarrow B_1 \mid B_2 \mid B_1 \ \&\& \ B_2 \mid \text{id1} \ \text{rel} \ \text{id2} \mid \text{true} \mid \text{false}$

According to your SDD what code will be generated for the following

expression? $\text{if } (x < 100 \mid x > 200 \ \&\& \ x \neq y) \ x = 0$

[2008. Marks: 6 + 2]

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.\text{next} = \text{newlabel}()$ $P.\text{code} = S.\text{code} \parallel \text{label}(S.\text{next})$
$S \rightarrow \text{assign}$	$S.\text{code} = \text{assign}.\text{code}$
$S \rightarrow \text{if } (B) \ S_1$	$B.\text{true} = \text{newlabel}()$ $B.\text{false} = S_1.\text{next} = S.\text{next}$ $S.\text{code} = B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$
$S \rightarrow \text{if } (B) \ S_1 \ \text{else} \ S_2$	$B.\text{true} = \text{newlabel}()$ $B.\text{false} = \text{newlabel}()$ $S_1.\text{next} = S_2.\text{next} = S.\text{next}$ $S.\text{code} = B.\text{code}$ $\parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$ $\parallel \text{gen('goto' } S.\text{next})$ $\parallel \text{label}(B.\text{false}) \parallel S_2.\text{code}$
$S \rightarrow \text{while } (B) \ S_1$	$\text{begin} = \text{newlabel}()$ $B.\text{true} = \text{newlabel}()$ $B.\text{false} = S.\text{next}$ $S_1.\text{next} = \text{begin}$ $S.\text{code} = \text{label}(\text{begin}) \parallel B.\text{code}$ $\parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$ $\parallel \text{gen('goto' } \text{begin})$
$S \rightarrow S_1 \ S_2$	$S_1.\text{next} = \text{newlabel}()$ $S_2.\text{next} = S.\text{next}$ $S.\text{code} = S_1.\text{code} \parallel \text{label}(S_1.\text{next}) \parallel S_2.\text{code}$

$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow (B_1)$	$B_1.true = B.true$ $B_1.false = B.false$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto' B.true)$ $\parallel gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$

Code for the expression **if (x < 100 || x > 200 && x != y) x = 0:**

```

        if x < 100 goto L2
        goto L3
L3:    if x > 200 goto L4
        goto L1
L4:    if x != y goto L2
        goto L1
L2:    x = 0
L1:

```

6.2 Write a syntax-directed definition that generates three-address code for Boolean expression of the following grammar:

$E \rightarrow E_1 \text{ or } E_2 \mid E_1 \text{ and } E_2 \mid \text{not } E_1 \mid (E_1) \mid id_1 \text{ relop } id_2 \mid \text{true} \mid \text{false}$

Using your definition, generate code for the expression

m < n or p < q and s < t [In-course 2, 2008-2009. Marks: 6 + 3.5]

[This grammar is a subset of the grammar from the above question (6.1). Just replace B with E, || with or, && with and, ! with not, rel with relop in the following SDD.]

Code for the expression m < n or p < q and s < t :

```

        if m < n goto L1
        goto L2
L2:    if p < q goto L3
        goto L4
L3:    if s < t goto L5
        goto L6

```

6.3 Write down a SDT scheme with backpatching to generate 3-address code for the following grammar:

$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{while } E \text{ do } S \mid \text{do}$

	$\begin{aligned} S & \text{ while } E \mid \{L\} \mid A \ E \rightarrow E \text{ and } E \mid E \text{ or } E \mid \text{not } E \mid (E) \mid \\ & \text{id relop id} \mid \text{true} \mid \text{false} \end{aligned}$
--	--

$$L \rightarrow L; S \mid S$$

Construct the annotated parse tree for the code segment below:

```
while (i > j) and (k < m) do
    if a > b then s = s + 1 else s = s - 1
```

Assume the arithmetic statements “s = s + 1” and “s = s - 1” above are generated from the non-terminal A in the above grammar. [2003. Marks: 8 + 4]

[The grammars are similar to the following grammars:]

- 1) $S \rightarrow \text{if}(B) M S_1$ { $\text{backpatch}(B.\text{truelist}, M.\text{instr});$
 $S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist});$ }
- 2) $S \rightarrow \text{if}(B) M_1 S_1 N \text{ else } M_2 S_2$
{ $\text{backpatch}(B.\text{truelist}, M_1.\text{instr});$
 $\text{backpatch}(B.\text{falselist}, M_2.\text{instr});$
 $\text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$
 $S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist});$ }
- 3) $S \rightarrow \text{while } M_1 (B) M_2 S_1$
{ $\text{backpatch}(S_1.\text{nextlist}, M_1.\text{instr});$
 $\text{backpatch}(B.\text{truelist}, M_2.\text{instr});$
 $S.\text{nextlist} = B.\text{falselist};$
 $\text{emit}(\text{'goto' } M_1.\text{instr});$ }
- 4) $S \rightarrow \{ L \}$ { $S.\text{nextlist} = L.\text{nextlist};$ }
- 5) $S \rightarrow A ;$ { $S.\text{nextlist} = \text{null};$ }
- 6) $M \rightarrow \epsilon$ { $M.\text{instr} = \text{nextinstr};$ }
- 7) $N \rightarrow \epsilon$ { $N.\text{nextlist} = \text{makelist}(\text{nextinstr});$
 $\text{emit}(\text{'goto' } -);$ }
- 8) $L \rightarrow L_1 M S$ { $\text{backpatch}(L_1.\text{nextlist}, M.\text{instr});$
 $L.\text{nextlist} = S.\text{nextlist};$ }
- 9) $L \rightarrow S$ { $L.\text{nextlist} = S.\text{nextlist};$ }
- 1) $B \rightarrow B_1 \mid \mid M B_2$ { $\text{backpatch}(B_1.\text{falselist}, M.\text{instr});$
 $B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$
 $B.\text{falselist} = B_2.\text{falselist};$ }
- 2) $B \rightarrow B_1 \&\& M B_2$ { $\text{backpatch}(B_1.\text{truelist}, M.\text{instr});$
 $B.\text{truelist} = B_2.\text{truelist};$
 $B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist});$ }
- 3) $B \rightarrow ! B_1$ { $B.\text{truelist} = B_1.\text{falselist};$
 $B.\text{falselist} = B_1.\text{truelist};$ }
- 4) $B \rightarrow (B_1)$ { $B.\text{truelist} = B_1.\text{truelist};$
 $B.\text{falselist} = B_1.\text{falselist};$ }
- 5) $B \rightarrow E_1 \text{ rel } E_2$ { $B.\text{truelist} = \text{makelist}(\text{nextinstr});$
 $B.\text{falselist} = \text{makelist}(\text{nextinstr} + 1);$
 $\text{emit}(\text{'if' } E_1.\text{addr rel.op } E_2.\text{addr 'goto' } -);$
 $\text{emit}(\text{'goto' } -);$ }
- 6) $B \rightarrow \text{true}$ { $B.\text{truelist} = \text{makelist}(\text{nextinstr});$
 $\text{emit}(\text{'goto' } -);$ }
- 7) $B \rightarrow \text{false}$ { $B.\text{falselist} = \text{makelist}(\text{nextinstr});$
 $\text{emit}(\text{'goto' } -);$ }
- 8) $M \rightarrow \epsilon$ { $M.\text{instr} = \text{nextinstr};$ }

	$S \rightarrow E = E$ $E \rightarrow E + E \mid id \mid id[E]$ i. Write a syntax-directed translation scheme for the grammar to produce 3-address codes. ii. Draw the annotated parse tree for the statement: $x[i] = x[x[i]] + i$, according to your translation scheme. Assume the array $x[]$ is a 10 element array where each element occupies 2 bytes. [2007. Marks: 5 + 5]
6.5	Consider the following grammar: [2007, 2005. Marks: 8 + 6] $S \rightarrow \text{if } E \text{ then } S \mid \text{for } id = NUM \text{ to } NUM \text{ Step by } NUM \ S \mid \{L\}$ $\mid A \ E \rightarrow E \text{ or } E \mid id \text{ relop } id \mid id \text{ relop } NUM$ $L \rightarrow L; \ S \mid S$ i. Write down a syntax-directed translation scheme with backpatching to generate 3-address codes for the grammar. Assume A generates assignment statements (e.g. $a = a + 1$) that are represented by a single quad. The semantics of “for”: id will be initialized to the first NUM, incremented in each iteration by the amount specified by the third NUM up to the second NUM. The token NUM is a signed integer. ii. According to your translation scheme for the grammar in question (i), construct the annotated parse tree for the following code fragment. Assume the 3-address codes start from quad 100. <pre> for a = 1 to 10 step by 2 { if b > 10 or a < c then b = b - 1; c = c - b } a = a + 1 </pre>
6.6	Consider the following grammar: [2004. Marks: 8 + 6] $S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{for } (A; E; A) \ S \mid \{L\} \mid A$ $E \rightarrow E \text{ and } E \mid E \text{ or } E \mid id \text{ relop } id \mid id \text{ relop } NUM$ $L \rightarrow L; \ S \mid S$ i. Write down a syntax-directed translation scheme with backpatching to generate 3-address codes for the grammar. Assume A generates assignment statements (e.g. $a = a + 1$) that are represented by a single quad and “for” has the same semantics of C’s <i>for</i> loop. ii. According to your translation scheme for the grammar in question (i), construct the annotated parse tree for the following code fragment. Assume the priority of ‘and’ is greater than that of ‘or’ and the 3-address codes start from quad 100. <pre> if (b > 10 or a < d and a > c) then b = b - 1 else d = d + 1 a = a + 1 </pre> iii. According to your translation scheme for the grammar in question (i), construct the annotated parse tree for the following code fragment. Assume the priority of ‘and’ is greater than that of ‘or’ and the 3-address codes start from quad 20. [In-course 2, 2007. Marks: 5] <pre> a = a + 1; for (b = 0; b > a; b = b + 1) { if b < c and b > 10 then b = c - 1 else b = c - 2 } </pre>

	<code>c = 2;</code>
6.7	Consider the following grammar: [2006. Marks: 8 + 6] <code>S → if E then S repeat S until E {L} A</code> <code>E → E or E id relop id</code>

14

	<code>L → L; S S</code> i. Write down a syntax-directed translation scheme with backpatching to generate 3-address codes for the grammar. Assume <i>A</i> generates assignment statements (e.g. <code>a = a + 1</code>) that are represented by a single quad and <code>relop</code> represents any relational operator. ii. According to your translation scheme for the grammar in question (i), construct the annotated parse tree for the following code fragment. Assume the 3-address codes start from quad 10. <pre>repeat { a = a + 1; if b < 10 or a > b then b = b + 1 } until (a > c)</pre>
6.8	Consider the following grammar: [2006. Marks: 8 + 6] <code>S → if E then S repeat S until E {L} A</code> <code>E → E and E id relop id (E)</code> <code>L → L; S S</code> i. Add production to generate “break” and “continue” statements. Write down a syntax directed translation scheme with backpatching to generate 3-address codes for the modified grammar. Assume: <i>A</i> generates assignment statements (e.g. <code>a = a + 1</code>) that are represented by a single quad. “break” and “continue” statements have the semantics that those statements have in C language. ii. According to your translation scheme for the grammar in question (i), construct the annotated parse tree for the following code fragment. Assume the 3-address codes start from quad 100. <pre>repeat { if (a > b and a < c) then break; a = a - b } until (a > d) a = a + 1</pre>
6.9	The following grammar generates expressions formed by applying an arithmetic operator + to integer and real constants. When two integers are added, the resulting type is integer, otherwise, it is real: <code>E → E + T T</code> <code>T → num . num num</code> Give an SDD to determine the type of each sub-expression. [2006. Marks: 4]

15

CHAPTER 8

CODE GENERATION

8.1 Briefly describe the issues in the design of a code generator. [In-course 3, 2008-2009. Marks: 4]

Issues in the design of a code generator:

1. Input to the Code Generator

The input to the code generator is the intermediate representation of the source program produced by the front end, along with information in the symbol table that is used to determine the run-time addresses of the data objects denoted by the names in the IR.

The many choices for the IR include three-address representations such as quadruples, triples, indirect triples; virtual machine representations such as bytecodes and stack-machine code; linear representations such as postfix notation; and graphical representations such as syntax trees and DAG's.

2. The Target Program

The instruction-set architecture of the target machine has a significant impact on the difficulty of constructing a good code generator that produces high-quality machine code. The most common target-machine architectures are RISC (reduced instruction set computer), CISC (complex instruction set computer), and stack based.

A RISC machine typically has many registers, three-address instructions, simple addressing modes, and a relatively simple instruction-set architecture. In contrast, a CISC machine typically has few registers, two-address instructions, a variety of addressing modes, several register classes, variable-length instructions, and instructions with side effects.

3. Instruction Selection

The code generator must map the IR program into a code sequence that can be executed by the target machine. The complexity of performing this mapping is determined by a factors such as

- the level of the IR
- the nature of the instruction-set architecture
- the desired quality of the generated code.

If the IR is high level, the code generator may translate each IR statement into a sequence of machine instructions using code templates. Such statement-by-statement code generation, however, often produces poor code that needs further optimization. If the IR reflects some of the low-level details of the underlying machine, then the code generator can use this information to generate more efficient code sequences.

4. Register Allocation



5. Evaluation Order



8.2

Write short notes on the following: [2003. Marks: 2×4]

i. Basic blocks

ii. Peephole optimization

i. Basic blocks are maximal sequences of consecutive three-address instructions with the properties that

- a. The flow of control can only enter the basic block through the first instruction in the block. That is, there are no jumps into the middle of the block.
- b. Control will leave the block without halting or branching, except possibly at the last instruction in the block.



ii.

8.3

With the help of an example describe the “next-use” algorithm. [In-course 3, 2008-2009. Marks: 4.5]

- Input: A basic block B of three-address statement. Initially the symbol table shows all nontemporary variables in B as being live on exit.
- Output: At each statement $i: x = y + z$ in B, attach to I the liveness and next-use information of x, y, z .
- Method: Start at the last statement in B and scan backwards. At each statement $i: x = y + z$ in B, do
 - Attach to statement i the information currently found in the symbol table regarding the next use and liveness of x, y, z .
 - In the symbol table, set x to “not live” and “no next use” (i.e., “dead”)
 - In the symbol table, set y and z to “live” and the next uses of y and z to i .

Example:



Table:

Initial Step 1 Step 2 Step 3 Step 4 Step 5 Step 6 Step 7 Step 8 Step 9 Step 10 t1 D D D D D D D D D D
D D L(2) D t2 D D D D D D D L(5) L(5) L(5) D D t3 D D D D D D D D L(4) D D D t4 D D D D D
D L(5) D D D D t5 D D D D D D L(6) D D D D D t6 D D D D D L(7) D D D D D D t7 D D L(9) D
D D D D D D D a L(0) L(0) L(0) L(0) L(0) L(0) L(0) L(0) L(0) L(0) L(2) L(2) b L(0) L(0) L(0) L(0)
L(0) L(0) L(0) L(4) L(4) L(4) L(4) prod L(0) L(0) L(0) L(0) D L(6) L(6) L(6) L(6) L(6) L(6) i
L(0) L(10) D L(8) L(8) L(8) L(8) L(8) L(8) L(3) L(3) L(1)

8.4

Write down the heuristic for graph coloring. What options do we have when an interference graph is found not to be k-colorable when there are k registers in the target machine? How do we determine the cost of fixing that problem? [*In-course. Marks: 3 + 3*]

ALSO, Describe the heuristic used to color an interference graph. [*In-course. Marks:*



2.5]

Exercises

8.1	Describe the code generation algorithm for a 2-address machine. Generate the code for the following code segment according to the algorithm. [<i>In-course. Marks: 6 + 4</i>] <pre> T1 := a + c T2 := b * T1 T3 := T1 - T2 T4 := T3 a := T3 * T4 </pre>
-----	--

8.2	Consider a hypothetical machine with four registers R1, R2, R3, R4 and six addressing modes with the following costs. <u>Addressing Mode Cost</u> <u>Absolute Memory Address 1</u> <u>Register 0</u> <u>Literal 0</u> <u>Indirect Register 1</u> <u>Indirect Plus Address 1</u> <u>Double Indirect 2</u> Now use an efficient algorithm to generate code for the target machine from the following block of 3-address codes: <pre> t1 := a + b t2 := t1 * c t3 := t2 - t1 b := t3 </pre> Calculate the cost of generated code and compare with cost of code generated with naïve approach to code generation. [<i>2008. Marks: 8</i>]
8.3	Draw the flow graph for the following program: [<i>In-course 3, 2008-2009. Marks: 4</i>] <pre> begin prod := 0; i := 1; do begin prod := prod + a[i] * b[i]; i := i + 1; end while (i <= 20); end </pre>
8.4	Draw the flow graph for the following sequence of 3-address codes. [<i>2008. Marks: 3</i>] <pre> (1) i = 0 (2) t1 = 10 (3) t2 = i < t1 (9) if t4 <= 20 goto (5) (10) t6 = z[t5] </pre>

	<pre> (11) *(t7) = t6 (4) if False t2 goto (15) (12) t8 = 1 (5) t3 = 4 (13) i = i + t9 (6) t4 = t3 * i (14) goto (2) (7) t5 = a + t4 (15) return (8) if t5 >= 100 goto (15) </pre>
8.5	For the following code fragment, determine the next-use information (assume that all the variables are live and all temporaries are dead at the end of the block). [2008. Marks: 5] <pre> (1) t6 := 4 * i (6) a[t7] := t9 (2) x := a[t6] (7) t10 := 4 * j (3) t7 := 4 * i (8) b[t10] := x (4) t8 := 4 * j (9) goto (5) t9 := a[t8] </pre>
8.6	Consider the following block of 3-address code: <pre> t1 := z * x t2 := z + t1 y := t2 * z z := x + y t1 := z * x y := x / t1 </pre> Use the graph coloring algorithm for register allocation for the block of code given above.

Assume the number of registers is $R = 3$ and all temporaries are dead at the end of the block.
[2008. Marks: 6]

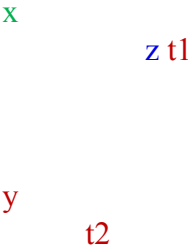
Liveness Information: Interference Graph with Coloring:

```
t1 := z * x {x, z}
t2 := z + t1 {x, z, t1}
y := t2 * z {x, z, t2}
```

```
z := x + y {x, y}
t1 := z * x {x, z}
y := x / t1 {x, z, t1}
```

{x, y, z}

Stack: t1, z, t2, x, y



CHAPTER 9
MACHINE-INDEPENDENT OPTIMIZATIONS

Theories

9.1

What do you understand by peephole optimizations? [In-course 3, 2008-2009. Marks:



4]

9.2	What is a copy statement? When can we eliminate copy statements? Give an example. [2007, 2004. Marks: 4] Assignments of the form $u = v$ are called <i>copy statements</i>. We can eliminate copy statements when there are common sub-expressions in statements. Example: 
9.3	What is meant by dead code? Give examples. [2007. Marks: 2] <i>Dead code</i> are statements that compute values that never get used. For example, suppose a variable <i>debug</i> is set to FALSE at various points in the program, and used in statements like <i>if (debug) print...</i> If copy propagation replaces <i>debug</i> by FALSE, then the <i>print</i> statement is dead because it cannot be reached. Hence, both the test and the print operation can be eliminated from the object code.

9.4	In order to generate optimized code, you have the options to do dead-code elimination, copy propagation, CSE (Common Sub-expression Elimination), global register allocation and instruction scheduling. In which order would you perform these operations? Justify your choice with proper reasoning. [2006. Marks: 5] 1. First of all, copy propagation should be performed. It would increase the possibility of finding common sub-expressions. 2. Then, CSE should be performed. 3. After these two steps, there would be a good possibility of existence of dead-code. So, at this stage, dead-code elimination should be performed. 4. Then, instruction scheduling might increase the chance of a more efficient register allocation. So, it should be performed. 5. Finally, Register allocation should be applied.
-----	--

9.5	Define UD- and DU-chains. What purposes do they serve? [2007, 2004. Marks: 4] UD-Chain: Purpose of UD-Chain: ! ! DU-Chain: Purpose of DU-Chain: ! !
9.6	What are the techniques to optimize a loop? Describe any one of them. [In-course 3, 2008-2009. Marks: 3.5] ALSO, What is the basic loop optimization technique? [2004. Marks: 1] The techniques to optimize a loop are as follows: 1. Code Motion 2. Induction-Variable Elimination 3. Reduction in Strength The basic loop optimization technique is <i>code motion</i>. If an expression is computed within a loop and it does not depend on variables that change in the loop, then it can be moved to just before the loop. Example:

	
9.7	Why would you be concerned to find whether a flow graph is reducible or not? [2007. Marks: 2] OR, For loop optimization, why are we interested in determining whether a flow graph is reducible or not? [2004. Marks: 2] 
9.8	Write down an algorithm for detecting loop invariant computations. [2007, 2005. Marks: 6]

Exercises

9.1

Consider the following fragment of intermediate code:

```

y = w
z = 4
v = y * y
u = z + 2
r = w ** 2 //this is exponentiation
t = r * v
s = u * t

```

Assume the only variable live at the exit is *s*. Show the result of applying constant propagation, algebraic simplification, common sub-expression elimination, constant folding, copy propagation and dead code elimination as much as possible to this code. You should explain the changes in each step. [2007, 2005. Marks: 6]

```

y = w
z = 4
v = y * y
  = w * w [Copy Propagation]
u = z + 2
  = 4 + 2 [Constant Propagation]
  = 6 [Constant Folding]
r = w ** 2
  = w * w [Algebraic Simplification]
  = v [Common Sub-expression Elimination]

```

23

```

t = r * v
  = v * v [Copy Propagation]
s = u * t
  = 6 * t [Constant Propagation]

```

After dead-code elimination:

```

v = w * w
t = v * v
s = 6 * t

```

9.2

Consider the following fragment of intermediate code:

```

w = 2
u = z
y = w + 1
v = y * y
r = v ** 2 //this is exponentiation
t = u * u
s = u * t
x = y * y

```

Assume the only variables live at the exit are *s*, *x*. Show the result of applying constant propagation, algebraic simplification, common sub-expression elimination, constant folding, copy propagation and dead code elimination as much as possible to this code. You should explain the changes in each step. [In-course. Marks: 4]

```

w = 2
u = z
y = w + 1
  = 2 + 1 [Constant Propagation]
  = 3 [Constant Folding]
v = y * y
  = 3 * 3 [Constant Propagation]
  = 9 [Constant Folding]
r = v ** 2
  = v * v [Algebraic Simplification]
  = 9 * 9 [Constant Propagation]
  = 81 [Constant Folding]
t = u * u
  = z * z [Copy Propagation]
s = u * t
  = z * t [Copy Propagation]
x = y * y
  = 3 * 3 [Constant Propagation]
  = 9 [Constant Folding]

```

After dead-code elimination:

```

t = z * z
s = z * t
x = 9

```

9.3

For the following code fragment, list all the dependencies between statements and draw the dependency graph. [2006. Marks: 3]

```

(1)          j = 4
(2)          k = j + 1
(3)          j = 6
(4)          m = k * j
(5)          m = m + 2
(6)          k = j + 1
(7)          j = k + j

```

	List of dependencies between statements: (2) depends on (1) for value of j. (4) depends on both (2) and (3) for values of k and j respectively. (5) depends on (4) for value of m.
--	---

	4. (6) depends on (3) for value of j. 5. (7) depends on both (3) and (6) for values of j and k respectively. Dependency Graph: Let, for statements A and B, denotes that A is dependent on B. <pre> A B 1 2 3 7 4 5 6 </pre>
9.4	Consider the following code fragment. [2003. Marks: 3 + 2 + 7] <pre> begin for i := 1 to n do for j := 1 to n do begin c[i,j] := 0; for k := 1 to n do c[i,j] = c[i,j] + a[i,k] * b[k,j]; end end end end </pre> i. Assume a, b and c are allocated static storage and there are 2 bytes per word in a byte addressable memory. Produce three-address code for the code fragment. ii. Construct the flow graph from the three address statement. iii. Optimize the code by eliminating common sub-expressions, eliminating dead-code and different loop optimization techniques.
9.5	Consider the following code segment. [2006. Marks: 3 + 5 + 4] <pre> (1) i = b + c (2) b = 10 (3) k = 9 (4) a = b + c (10) goto (13) (11) g = -k (12) f = d + 4 (13) c = 25 </pre>

	(5) d = e + f (6) k = j + 1 (7) if p > 10 goto (9) (8) goto (11) (9) e = 5 (14) h = e + 10 (15) j = b + j (16) if q > 10 goto (4) (17) exit i. Draw the control flow graph (CFG). ii. Perform global CSE and draw the CFG for the code that results from CSE. You need not to show the computation for finding available expressions. Only show the available expressions at the input of each basic block. iii. Find all natural loops and identify the loop invariant statements. Which statements are safe to be moved to the loop's pre-header and why?
9.6	Consider the code segment below: [2004. Marks: 3 + 6] (1) m = 5 (2) f = 0 (3) g = 1 (4) if m < 10 goto (6) (5) return m (6) i = 2 (7) if i < m goto (9) (8) return g (9) h = f - g (10) g = f (11) f = h (12) i = i + 1 (13) goto (7) i. Construct a flow graph.

	ii. Find the live variables at the end of each block.
--	--

	m = 5	use B ₁ = {}
B ₁	f = 0	def B ₁ = {m, f, g}
	g = 1	
	if m < 10 goto B ₃	
		use B ₂ = {m}
B ₂	return m	def B ₂ = {}
		use B ₃ = {}
B ₃	i = 2	def B ₃ = {i}
		use B ₄ = {i, m}
B ₄	if i < m goto B ₆	def B ₄ = {}
		use B ₅ = {g}
	return g	def B ₅ = {}
B ₅		
	h = f - g	
	g = f	use B ₆ = {f, g, i}
B ₆	f = h	def B ₆ = {f, g, h, i}
	i = i + 1	
	goto B ₄	

$IN[B]^0 \text{ OUT}[B]^1 IN[B]^1 \text{ OUT}[B]^2 IN[B]^2 \text{ OUT}[B]^3 IN[B]^3$
B₁ { } { f, g, m } { } { f, g, m } { } {
f, g, m } { } **B₂** { } { } { m } { } { m } { } { m } **B₃** { } { f, g, i, m } { f, g, m } { f, g, i, m } { f,
g, m } { f, g, i, m } { f, g, m } **B₄** { } { f, g, i } { f, g, i, m } { f, g, i, m } { f, g, i, m } { f, g, i, m }
{ f, g, i, m } **B₅** { } { } { g } { } { g } { } { g } **B₆** { } { } { f, g, i } { f, g, i, m } { f, g, i, m } {
f, g, i, m } { f, g, i, m }

∴ **Live variables after each block:**

B₁: { f, g, m }
B₂: { }
B₃: { f, g, i, m }
B₄: { f, g, i, m }
B₅: { }
B₆: { f, g, i, m }

[Points to be noted from this answer:

1. There is no edge from the block containing **return m** to the following block, because after returning from a block, the program never flows below in the block. The same case holds for **return g**, too. This is according to the rule of putting an edge in flow graph (from page 529,

	bullet point no. 2). 2. As there is no edge outgoing from <i>return m</i>, hence its OUT is empty. However, its IN is not empty as it <i>uses m</i>. 3. Block B₄ should be counted – even though there is no variable assignment statement. That’s because the compiler must know to before comparing $i < m$ whether both are live or not. And after the end of the block, should <i>i</i> and <i>m</i> be dead and discarded or not. Similarly, <i>m</i> in B₁ should also be counted. 4. <i>m</i> is not <i>used</i> in B₁ as can be mistakenly assumed from the <i>if</i> instruction. That’s because <i>m</i> is
--	--

	defined <i>before</i> it is used in this block. From the definition of <i>use_B</i> (from page 609, point 2), <i>m</i> is not <i>used</i>. Similarly, <i>h</i> is not <i>used</i> in B₆. (Also see page 609, the paragraph after <i>Example 9.13</i> for further clarification.) 5. Although usually live variable analysis includes a block containing EXIT, in this particular case, there should be no exit block. That’s because the last block B₆ includes as its last statement an <i>unconditional</i> jump – <i>goto B₃</i>, after executing which the program control will always flow to B₄ and never to any other block.]
9.7	Consider the following sequence of 3-address codes: [In-course. Marks: 4 + 6] (1) <pre> (1) e = e - b (2) d = a * c (3) if e < d goto (1) (4) i = e + f (5) j = a + b (6) c = c * 2 (7) if c > d goto (9) (8) goto (11) (9) g = a * c (10) goto (13) (11) i = d * d (12) j = c + 1 (13) if i > j goto (5) (14) exit </pre> i. Draw the flow graph. ii. Compute live variables at the end of each block using the iterative solution to dataflow equations for live variable analysis.

9.8	Consider the following sequence of 3-address codes: [<i>In-course. Marks: 2 + 6 + 2</i>] (1) <pre> c = a + b (8) goto (11) (2) d = a * c (9) g[i] = a * c (3) e = d * d (10) goto (13) (4) i = 1 (11) g[i] = d * d (5) f[i] = a + b (12) i = i + 1 (6) c = c * 2 (13) if i > 10 goto (5) (7) if c > d goto (9) (14) exit </pre> i. Draw the flow graph. ii. Compute the available expressions at the beginning of each block using the iterative solution to dataflow equations for available expressions. iii. Draw the flow diagram after global CSE.
9.9	Consider the following sequence of 3-address codes: [<i>2007. Marks: 4 + 6</i>] <pre> (1) a = a - d (8) e = c - a (2) f = b * d (9) if e > 10 goto (3) (3) c = a + b (10) goto (13) (4) d = c - a (11) a = b * d (5) if d > x goto (7) </pre>

	<pre> (12) b = a - d (6) d = b * d (13) if b > 10 goto (1) (7) b = a + b (14) exit </pre> i. Draw the flow graph. ii. Compute live variables at the end of each block using the iterative solution to dataflow equations for live variable analysis. iii. Show the execution of the algorithm solving the data flow equations set up for available expressions. [<i>In-course. Marks: 6</i>]
9.10	Consider the following sequence of 3-address codes: [2005. Marks: 2 + 5] <pre> (1) i = m - 1 (7) goto (9) (2) j = n (8) a = u2 (3) a = u1 (9) i = u3 (4) i = i + 1 (10) if u3 > 0 goto (4) (5) j = j - 1 (11) exit (6) if i > j goto (8) </pre>

- i. Draw the flow graph.
- ii. Find the definitions reaching the end of each block by iteratively solving the dataflow equations for reaching definitions.



$\therefore$ Definitions reaching at the end of each block:

B1: $\{ d_1, d_2, d_3 \}$

B2: $\{ d_3, d_4, d_5, d_6 \}$

B3: $\{ d_4, d_5, d_6 \}$

B4: $\{ d_3, d_5, d_6, d_7 \}$

9.11

Consider the following flow graph: [In-course 3, 2008-2009. Marks: 4 + 2]

- i. Compute UD and DU-chain for the flow graph.
 - ii. Compute live variables at the end of each block of the flow graph.
- i. UD-Chain:

$[\cdot, \cdot]$
 $[\cdot]$

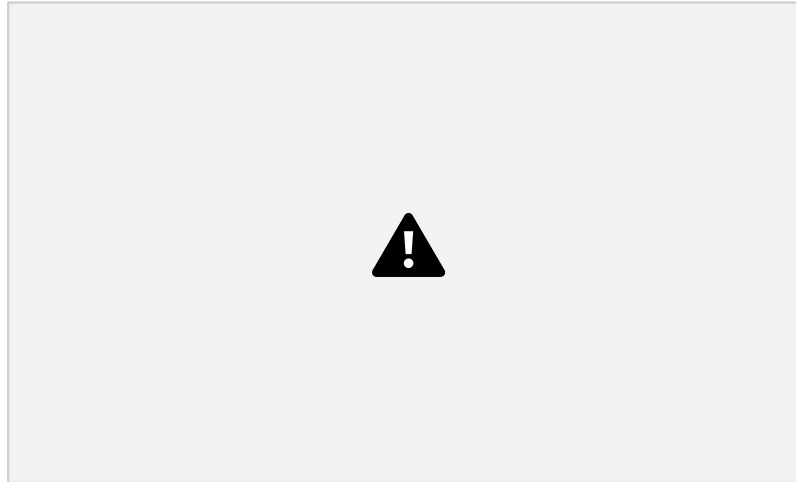
DU-Chain:

$[\cdot]_{[\cdot]}$

$[\cdot]$

9.12

Consider the following flow graph: [2008. Marks: 4 + 2 + 2 + 4 + 4]



- i. Compute UD and DU-chain for the above flow graph.
- ii. Compute live variables at the end of each block of the flow graph.
- iii. Compute available expressions for the flow graph.
- iv. Is any constant folding possible in the flow graph? If so, do it.
- v. Are there any common sub-expressions in the flow graph? If so, do it.

9.13

Consider the CFG below where only definition ($v = \dots$) and use ($\dots = v$) of the interesting variables are shown.

- i. List the webs as $web_i = \{\text{list of statements } S_{ks} \text{ in } web_i\}$.
- ii. Draw the interference graph.
- iii. If coloring is possible and if you have three registers R_0, R_1, R_2 , identify for each node the allocated register allocation. Assume the variable k cannot be allocated to register R_2 . Otherwise, which node would you choose to spill and why? [2006. Marks: 2 + 3 + 3]

9.14

Consider the CFG below where only definition ($v = \dots$) and use ($\dots = v$) of the interesting variables are shown.

- i. List the webs as $\text{web}_i = \{\text{list of statements } S_{ks} \text{ in } \text{web}_i\}$.**
- ii. Draw the interference graph.**
- iii. If you have three registers, show the code after register allocation (if coloring is possible). [2005. Marks: 2 + 3 + 3]**